Universidad Politécnica de Atlautla

Actividad 2.4. Algoritmos de búsqueda ciega

"Implementación y comparación de BFS y DFS para resolver un laberinto"

Alumno: Castro Soriano Ronaldo Malcom.

Docente: Ing. Garcia Lara Javier.

25 de octubre de 2026.

1. Tema y planteamiento del problema

El objetivo es encontrar una ruta desde la entrada de un laberinto hasta su salida. El laberinto contiene pasillos transitables y muros que no pueden cruzarse. Cada movimiento entre dos posiciones adyacentes tiene costo uno. Se comparan dos algoritmos de búsqueda no informada: búsqueda en anchura (Breadth-First Search, BFS) y búsqueda en profundidad (Depth-First Search, DFS). BFS explora por niveles y encuentra una ruta de menor número de pasos en un grafo no ponderado; DFS profundiza por una rama antes de retroceder y no garantiza la ruta más corta (Cormen et al., 2009; Russell & Norvig, 2021).

2. Datos y representación del laberinto

El programa genera un laberinto reproducible con semilla 23. La matriz del laberinto mide 19 filas por 27 columnas. El grafo tiene 252 nodos transitables y 270 aristas. El inicio es la coordenada (1, 1) y la meta es la coordenada (17, 25). Los datos del laberinto son generados por el programa con finalidad didáctica.

Representación formal: G = (V, E). Cada posición transitable es un nodo de V. Se agrega una arista no dirigida entre dos nodos si sus posiciones están una al lado de la otra en dirección horizontal o vertical y no hay un muro entre ellas. El grafo es no ponderado porque todos los movimientos valen una unidad.

Estado: posición actual (fila, columna). **Acciones:** mover a la derecha, abajo, izquierda o arriba, siempre que la posición vecina sea transitable. **Costo:** una unidad por movimiento. **Meta:** llegar a la salida G. **Solución:** una secuencia contigua de nodos desde S hasta G.

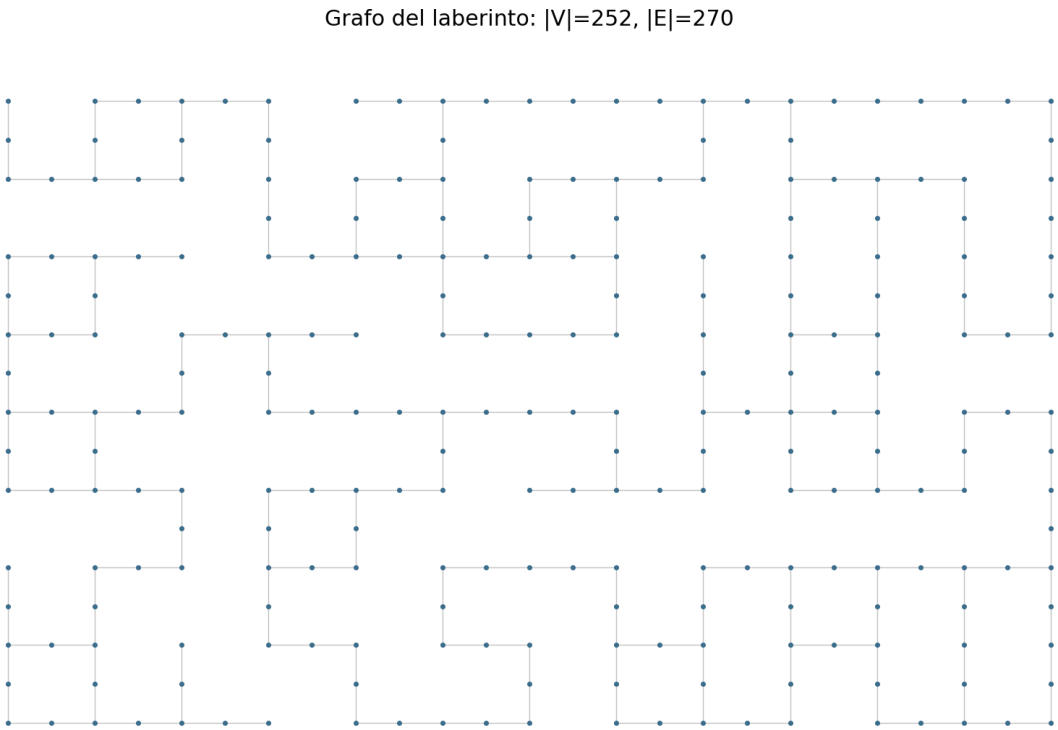
3. Procedimiento de búsqueda

BFS. Utiliza una cola FIFO. Al visitar un nodo, registra sus vecinos no descubiertos y los agrega al final de la cola. Como procesa los nodos por profundidad creciente, produce el camino más corto cuando todas las aristas tienen el mismo costo.

DFS. Utiliza una pila LIFO. Agrega los vecinos a la pila y procesa primero el último agregado. Puede encontrar una ruta válida sin garantizar que tenga el menor número de movimientos.

Ambas implementaciones registran el predecesor de cada nodo para reconstruir el camino cuando se alcanza la meta. Los nodos expandidos se contabilizan como los nodos extraídos de la cola o de la pila y procesados por el algoritmo, incluyendo la meta cuando es extraída. El orden de movimientos se fija para que las pruebas sean repetibles.

4. Diagrama del grafo

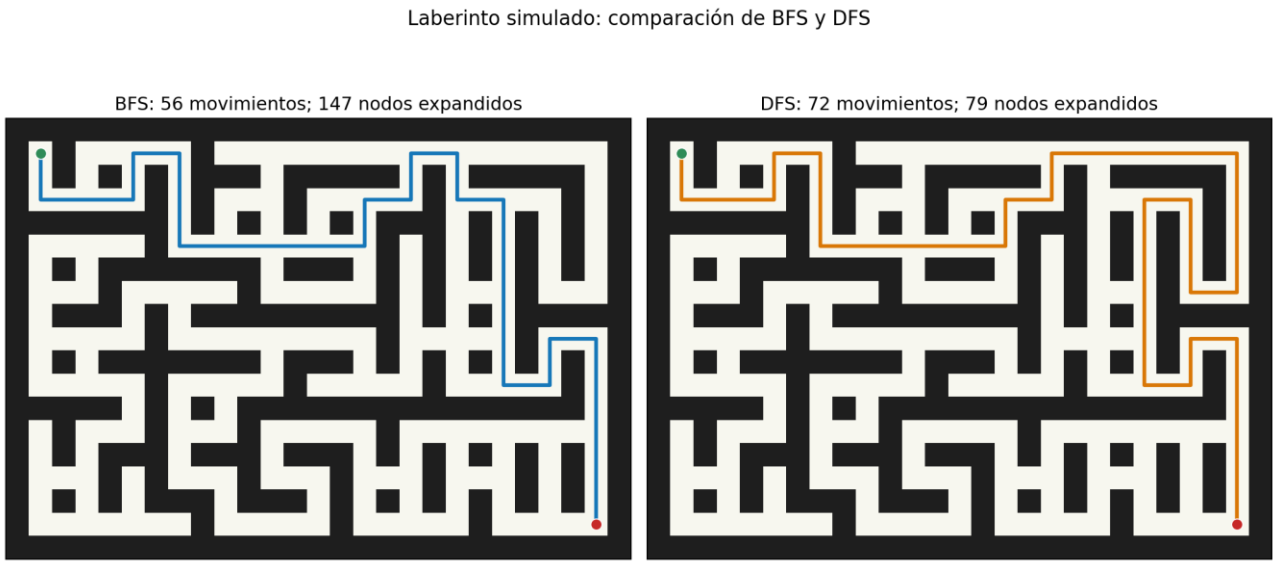


El grafo se deriva directamente de los pasillos del laberinto. Cada nodo representa una posición transitable y cada arista un movimiento legal de una sola casilla. La cantidad de nodos y aristas corresponde a la instancia utilizada en el programa.

5. Soluciones encontradas y comparación

Algoritmo	Movimientos	Nodos expandidos	Tiempo mediano
BFS	56	147	0.1177 ms
DFS	72	79	0.0690 ms

El tiempo es la mediana de 30 ejecuciones medidas en el entorno donde se generó el reporte. Los tiempos pueden cambiar de un equipo a otro; por su magnitud pequeña, no deben usarse como una medida universal del rendimiento.



El camino azul corresponde a BFS y contiene el menor número de movimientos. El camino naranja corresponde a DFS; en esta ejecución es más largo, aunque expandió menos nodos y registró un tiempo mediano inferior. Esos resultados dependen del orden de vecinos y de la estructura concreta del laberinto.

6. Análisis de complejidad temporal y espacial.

Sea V el número de posiciones transitables y E el número de conexiones entre posiciones. Con una representación por listas de adyacencia, BFS tiene complejidad temporal O(V + E), porque cada nodo se descubre y procesa como máximo una vez y cada arista se examina un número constante de veces. Su complejidad espacial es O(V), por la cola, el conjunto de descubiertos y los predecesores (Cormen et al., 2009).

DFS también tiene complejidad temporal O(V + E), pues cada nodo y arista se examina como máximo un número constante de veces. Su complejidad espacial es O(V), debido a la pila, el conjunto de descubiertos y los predecesores. Aunque ambos tienen la misma cota asintótica, el número efectivo de nodos visitados puede variar según el grafo y el orden de expansión.

Para esta instancia se obtuvieron V = 252 y E = 270. Como el laberinto es una cuadrícula y cada posición puede conectarse con un máximo de cuatro vecinos, E es O(V); por ello, para este tipo de grafo la cota O(V + E) se simplifica a O(V). La búsqueda conserva solo los datos del grafo, la frontera y los predecesores; no necesita almacenar todos los caminos posibles.

7. Conclusiones

Los dos algoritmos encontraron una solución válida. BFS fue el método adecuado cuando la prioridad es garantizar el menor número de movimientos en un laberinto sin pesos, pues encontró un camino de 56 pasos frente a los 72 de DFS. BFS expandió más nodos en esta prueba. DFS encontró la salida con 79 nodos expandidos frente a 147 de BFS y tuvo un tiempo mediano menor en el entorno de prueba, pero su ruta no fue óptima. Por ello, BFS se recomienda cuando importa la ruta más corta; DFS puede ser útil cuando basta encontrar alguna solución y se acepta que pueda ser más larga.

8. Referencias APA 7.ª edición

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

NetworkX Developers. (s. f.). *Traversal algorithms*. NetworkX documentation. https://networkx.org/documentation/stable/reference/algorithms/traversal.html

Python Software Foundation. (s. f.). *time — Time access and conversions*. Python documentation. https://docs.python.org/3/library/time.html

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Anexo A. Código Python completo: actividad 2.4

El siguiente anexo reproduce el archivo Python incluido por separado. El código está como texto editable; para ejecutarlo se recomienda usar el archivo .py original.

```
"""Actividad 2.4. Algoritmos de búsqueda ciega BFS y DFS en un laberinto.


El programa genera un laberinto reproducible, lo representa como un grafo no
ponderado y compara búsqueda en anchura (BFS) con búsqueda en profundidad (DFS).


Instalación:
    python -m pip install matplotlib networkx


Ejecución:
    python actividad_2_4_bfs_dfs.py
"""
from __future__ import annotations


import json
import random
import statistics
import time
from collections import deque
from pathlib import Path


import matplotlib.pyplot as plt
import networkx as nx
import numpy as np


BASE_DIR = Path(__file__).resolve().parent
FIG_DIR = BASE_DIR / "figuras"
FIG_DIR.mkdir(parents=True, exist_ok=True)


FILAS_CELDAS = 9
COLUMNAS_CELDAS = 13
SEMILLA_LABERINTO = 23
# Orden fijo de movimientos: derecha, abajo, izquierda, arriba.
MOVIMIENTOS = [(0, 1), (1, 0), (0, -1), (-1, 0)]



def generar_laberinto(filas: int, columnas: int, semilla: int) -> list[list[str]]:
    """Crea un laberinto aleatorio reproducible y abre algunos atajos adicionales."""
    rng = random.Random(semilla)
    alto, ancho = 2 * filas + 1, 2 * columnas + 1
    matriz = [["#" for _ in range(ancho)] for _ in range(alto)]
    inicio_celda = (0, 0)
    visitadas = {inicio_celda}
    pila = [inicio_celda]
    matriz[1][1] = " "


    # Generación por retroceso: cada celda se conecta con una celda vecina no visitada.
    while pila:
        fila, columna = pila[-1]
        candidatos = []
        for df, dc in MOVIMIENTOS:
            nf, nc = fila + df, columna + dc
            if 0 <= nf < filas and 0 <= nc < columnas and (nf, nc) not in visitadas:
                candidatos.append((nf, nc, df, dc))
        if not candidatos:
            pila.pop()
            continue


        nf, nc, df, dc = rng.choice(candidatos)
        matriz[2 * fila + 1 + df][2 * columna + 1 + dc] = " "
        matriz[2 * nf + 1][2 * nc + 1] = " "
        visitadas.add((nf, nc))
        pila.append((nf, nc))


    # Se abren algunos muros entre celdas para crear rutas alternativas.
    muros_candidatos = []
    for fila in range(1, alto - 1):
        for columna in range(1, ancho - 1):
            if matriz[fila][columna] != "#":
                continue
            if fila % 2 == 1 and columna % 2 == 0:
                conecta = (matriz[fila][columna - 1] == " " and
                           matriz[fila][columna + 1] == " ")
            elif fila % 2 == 0 and columna % 2 == 1:
```

```
                conecta = (matriz[fila - 1][columna] == " " and
                           matriz[fila + 1][columna] == " ")
            else:
                conecta = False
            if conecta:
                muros_candidatos.append((fila, columna))

    rng.shuffle(muros_candidatos)
    aperturas = max(1, int(len(muros_candidatos) * 0.20))
    for fila, columna in muros_candidatos[:aperturas]:
        matriz[fila][columna] = " "

    matriz[1][1] = "S"
    matriz[alto - 2][ancho - 2] = "G"
    return matriz



def construir_grafo(matriz: list[list[str]]) -> nx.Graph:
    """Convierte cada espacio transitable en un nodo y cada movimiento en una arista."""
    grafo = nx.Graph()
    alto, ancho = len(matriz), len(matriz[0])
    transitables = {" ", "S", "G"}
    for fila in range(alto):
        for columna in range(ancho):
            if matriz[fila][columna] not in transitables:
                continue
            nodo = (fila, columna)
            grafo.add_node(nodo)
            for df, dc in MOVIMIENTOS:
                nf, nc = fila + df, columna + dc
                if (0 <= nf < alto and 0 <= nc < ancho and
                        matriz[nf][nc] in transitables):
                    grafo.add_edge(nodo, (nf, nc), costo=1)
    return grafo



def ordenar_vecinos(grafo: nx.Graph, nodo: tuple[int, int]) -> list[tuple[int, int]]:
    """Devuelve vecinos siguiendo el mismo orden de movimientos para repetir la prueba."""
    vecinos = set(grafo.neighbors(nodo))
    ordenados = []
    for df, dc in MOVIMIENTOS:
        candidato = (nodo[0] + df, nodo[1] + dc)
        if candidato in vecinos:
            ordenados.append(candidato)
    return ordenados



def reconstruir_camino(
    padre: dict[tuple[int, int], tuple[int, int] | None],
    meta: tuple[int, int],
) -> list[tuple[int, int]]:
    """Reconstruye la ruta desde la meta usando los predecesores registrados."""
    if meta not in padre:
        return []
    camino = []
    actual: tuple[int, int] | None = meta
    while actual is not None:
        camino.append(actual)
        actual = padre[actual]
    camino.reverse()
    return camino



def busqueda_bfs(
    grafo: nx.Graph,
    inicio: tuple[int, int],
    meta: tuple[int, int],
) -> dict:
    """BFS: explora primero todos los nodos de menor profundidad."""
    cola = deque([inicio])
    descubiertos = {inicio}
    padre = {inicio: None}
    orden_expansion = []

    while cola:
        actual = cola.popleft()
        orden_expansion.append(actual)
        if actual == meta:
            break
        for vecino in ordenar_vecinos(grafo, actual):
```

```
            if vecino not in descubiertos:
                descubiertos.add(vecino)
                padre[vecino] = actual
                cola.append(vecino)

    camino = reconstruir_camino(padre, meta)
    return {
        "camino": camino,
        "nodos_expandidos": len(orden_expansion),
        "orden_expansion": orden_expansion,
        "encontrado": bool(camino),
    }



def busqueda_dfs(
    grafo: nx.Graph,
    inicio: tuple[int, int],
    meta: tuple[int, int],
) -> dict:
    """DFS: sigue una rama lo más profundo posible antes de retroceder."""
    pila = [inicio]
    descubiertos = {inicio}
    padre = {inicio: None}
    orden_expansion = []

    while pila:
        actual = pila.pop()
        orden_expansion.append(actual)
        if actual == meta:
            break

        # Invertir el orden al insertar hace que el primer movimiento preferido
        # (derecha, luego abajo, izquierda y arriba) salga primero de la pila.
        for vecino in reversed(ordenar_vecinos(grafo, actual)):
            if vecino not in descubiertos:
                descubiertos.add(vecino)
                padre[vecino] = actual
                pila.append(vecino)

    camino = reconstruir_camino(padre, meta)
    return {
        "camino": camino,
        "nodos_expandidos": len(orden_expansion),
        "orden_expansion": orden_expansion,
        "encontrado": bool(camino),
    }



def medir_tiempo_mediano(
    funcion,
    grafo: nx.Graph,
    inicio: tuple[int, int],
    meta: tuple[int, int],
    repeticiones: int = 30,
) -> tuple[dict, float]:
    """Mide la mediana de varias ejecuciones para reducir ruido de temporización."""
    tiempos = []
    resultado = None
    for _ in range(repeticiones):
        t0 = time.perf_counter()
        resultado = funcion(grafo, inicio, meta)
        tiempos.append(time.perf_counter() - t0)
    return resultado, statistics.median(tiempos) * 1000.0



def dibujar_comparacion(
    matriz: list[list[str]],
    bfs: dict,
    dfs: dict,
    ruta: Path,
) -> None:
    """Dibuja el laberinto y superpone la ruta encontrada por cada algoritmo."""
    alto, ancho = len(matriz), len(matriz[0])
    imagen = np.ones((alto, ancho, 3), dtype=float)
    for fila in range(alto):
        for columna in range(ancho):
            if matriz[fila][columna] == "#":
                imagen[fila, columna] = (0.12, 0.12, 0.12)
            else:
                imagen[fila, columna] = (0.97, 0.97, 0.94)
```

```
    fig, ejes = plt.subplots(1, 2, figsize=(12, 6))
    paneles = [(ejes[0], bfs, "BFS", "#1677B8"),
               (ejes[1], dfs, "DFS", "#D97706")]
    for ax, resultado, titulo, color in paneles:
        ax.imshow(imagen, interpolation="nearest")
        camino = resultado["camino"]
        if camino:
            ax.plot([n[1] for n in camino], [n[0] for n in camino],
                    color=color, linewidth=2.5, solid_capstyle="round")
            ax.scatter([camino[0][1]], [camino[0][0]], s=55, c="#2E8B57",
                       edgecolors="white", linewidths=0.7, zorder=5)
            ax.scatter([camino[-1][1]], [camino[-1][0]], s=55, c="#C62828",
                       edgecolors="white", linewidths=0.7, zorder=5)
        ax.set_title(f"{titulo}: {len(camino) - 1} movimientos; "
                     f"{resultado['nodos_expandidos']} nodos expandidos")
        ax.set_xticks([])
        ax.set_yticks([])
        ax.set_xlim(-0.5, ancho - 0.5)
        ax.set_ylim(alto - 0.5, -0.5)
    fig.suptitle("Laberinto simulado: comparación de BFS y DFS", fontsize=13)
    fig.tight_layout()
    fig.savefig(ruta, dpi=180, bbox_inches="tight")
    plt.close(fig)


def guardar_grafo(grafo: nx.Graph, ruta: Path) -> None:
    """Guarda una vista del grafo de pasillos; cada nodo es una posición transitable."""
    posiciones = {n: (n[1], -n[0]) for n in grafo.nodes}
    fig, ax = plt.subplots(figsize=(10, 6.3))
    nx.draw_networkx_edges(grafo, posiciones, ax=ax, width=0.55,
                           edge_color="#A0A0A0", alpha=0.65)
    nx.draw_networkx_nodes(grafo, posiciones, ax=ax, node_size=8,
                           node_color="#3B6D8C", linewidths=0)
    ax.set_title(f"Grafo del laberinto: |V|={grafo.number_of_nodes()}, "
                 f"|E|={grafo.number_of_edges()}")
    ax.set_axis_off()
    fig.tight_layout()
    fig.savefig(ruta, dpi=180, bbox_inches="tight")
    plt.close(fig)


def serializar_camino(camino: list[tuple[int, int]]) -> list[list[int]]:
    """Convierte coordenadas de nodos a listas para guardarlas como JSON."""
    return [[fila, columna] for fila, columna in camino]


def main() -> None:
    """Genera el laberinto, ejecuta BFS/DFS, mide y guarda todos los resultados."""
    matriz = generar_laberinto(FILAS_CELDAS, COLUMNAS_CELDAS, SEMILLA_LABERINTO)
    grafo = construir_grafo(matriz)
    inicio = (1, 1)
    meta = (len(matriz) - 2, len(matriz[0]) - 2)

    bfs, tiempo_bfs = medir_tiempo_mediano(busqueda_bfs, grafo, inicio, meta)
    dfs, tiempo_dfs = medir_tiempo_mediano(busqueda_dfs, grafo, inicio, meta)
    dibujar_comparacion(matriz, bfs, dfs, FIG_DIR / "laberinto_bfs_dfs.png")
    guardar_grafo(grafo, FIG_DIR / "grafo_laberinto.png")

    resultados = {
        "nota": "Laberinto generado de forma determinista con datos didácticos.",
        "semilla": SEMILLA_LABERINTO,
        "filas_matriz": len(matriz),
        "columnas_matriz": len(matriz[0]),
        "nodos_grafo": grafo.number_of_nodes(),
        "aristas_grafo": grafo.number_of_edges(),
        "inicio": list(inicio),
        "meta": list(meta),
        "repeticiones_tiempo": 30,
        "BFS": {
            "encontrado": bfs["encontrado"],
            "camino": serializar_camino(bfs["camino"]),
            "movimientos": len(bfs["camino"]) - 1,
            "nodos_expandidos": bfs["nodos_expandidos"],
            "tiempo_mediano_ms": tiempo_bfs,
        },
        "DFS": {
            "encontrado": dfs["encontrado"],
            "camino": serializar_camino(dfs["camino"]),
            "movimientos": len(dfs["camino"]) - 1,
            "nodos_expandidos": dfs["nodos_expandidos"],
```

```
            "tiempo_mediano_ms": tiempo_dfs,
        },
    }
    salida_json = BASE_DIR / "resultados_2_4.json"
    salida_json.write_text(json.dumps(resultados, indent=2, ensure_ascii=False), encoding="utf-8")

    # Guardar el laberinto como texto facilita revisar su entrada y reproducirla.
    (BASE_DIR / "laberinto.txt").write_text(
        "\n".join("".join(fila) for fila in matriz), encoding="utf-8"
    )

    print("ACTIVIDAD 2.4 - RESULTADOS")
    print(f"Grafo: V={grafo.number_of_nodes()} nodos; E={grafo.number_of_edges()} aristas")
    print(f"Inicio={inicio}; meta={meta}")
    print(f"BFS: encontrado={bfs['encontrado']}; movimientos={len(bfs['camino']) - 1}; "
          f"expandidos={bfs['nodos_expandidos']}; mediana={tiempo_bfs:.4f} ms")
    print(f"DFS: encontrado={dfs['encontrado']}; movimientos={len(dfs['camino']) - 1}; "
          f"expandidos={dfs['nodos_expandidos']}; mediana={tiempo_dfs:.4f} ms")
    print(f"Figuras: {FIG_DIR}")
    print(f"Resultados: {salida_json}")


if __name__ == "__main__":
    main()
```

Declaración de uso de inteligencia artificial

Herramienta: ChatGPT, de OpenAI.

Prompt utilizado: “Implementa BFS y DFS en Python para resolver un laberinto; documenta el grafo, complejidades, camino solución, nodos expandidos, tiempos, comparación y conclusión.”

Uso: apoyo para estructurar el planteamiento, formalizar el problema, redactar explicaciones, construir y comentar el código Python y organizar los resultados.

Verificación: el programa fue ejecutado con la semilla fija del laberinto; se comprobaron la existencia de ambos caminos, la continuidad entre nodos y el conteo de movimientos, nodos expandidos y tiempos medianos sobre 30 ejecuciones.

Los datos generados para el ejemplo se identifican como simulados. La interpretación final debe ser revisada por el estudiante antes de entregar la actividad.